

Access S3 from a VPC

Melvin green

```
[ec2-user@ip-10-0-4-207 ~]$ sudo touch /tmp/test.txt
[ec2-user@ip-10-0-4-207 ~]$ aws s3 cp /tmp/test.txt s3://mel-vpc-project
upload: ../../tmp/test.txt to s3://mel-vpc-project/test.txt
[ec2-user@ip-10-0-4-207 ~]$ aws s3 ls s3://mel-vpc-project
2026-06-18 02:05:17      1598726 kari-shea-18AnrIwx50Y-unsplash.jpg
2026-06-18 02:05:23      1825778 lorenzo-herrera-p0j-mE6mGo4-unsplash.jpg
2026-06-18 03:08:26           0 test.txt
[ec2-user@ip-10-0-4-207 ~]$
```

Introducing Today's Project!

What is Amazon VPC?

Amazon VPC (Virtual Private Cloud) is AWS's foundational networking service, allowing users to provision a logically isolated section of the AWS cloud where they can define and control their own virtual network, including IP address ranges, subnets, route tables, and security settings.

How I used Amazon VPC in this project

In today's project, I launched a VPC with a public subnet and an EC2 instance, then used AWS CLI to directly access and manage an Amazon S3 bucket from within that instance, demonstrating how resources outside a VPC can be securely integrated with resources running inside it.

One thing I didn't expect in this project was...

One key takeaway from this project was realizing that access keys are required for EC2 instances and other applications to authenticate and gain access to my AWS environment, reinforcing the importance of secure credential management when connecting compute resources to AWS services.

This project took me...

This project took me roughly an hour and a half.

In the first part of my project...

Step 1 - Architecture set up

In this step, I will launch a VPC and an EC2 Instance.

Step 2 - Connect to my EC2 instance

In this step, I will connect to my EC2 instance and attempt to connect to an AWS service.

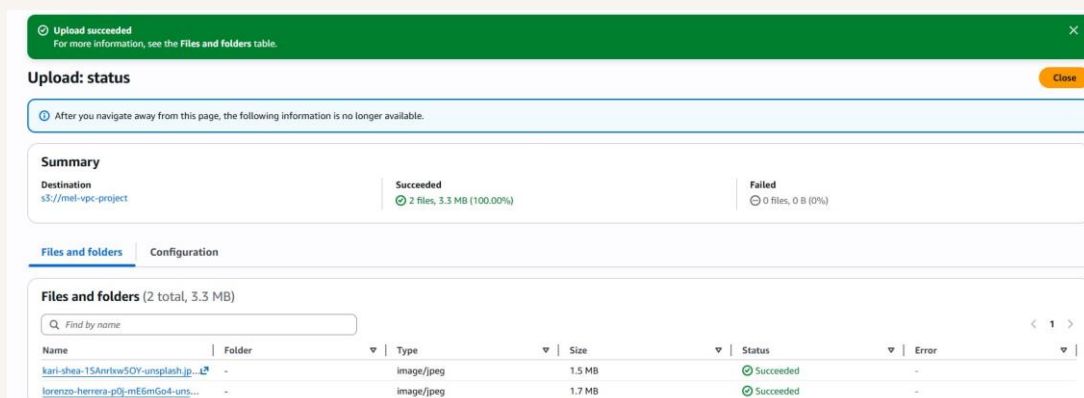
Step 3 - Set up access keys

In this step, I will set up an access key because this will give me the ability to access my AWS resources.

Architecture set up

In this step, I will launch a VPC using the 'VPC and More' option. This will create a VPC, a public subnet, and an internet gateway. Once those are created, I will create an EC2 instance.

I also set up my S3 bucket and uploaded two files from my local computer.



Running CLI commands

AWS CLI stands for AWS Command Line Interface, a tool that can be installed on a computer to interact with and control AWS services directly through commands rather than the web console. In this project, I had direct access to AWS CLI because it comes pre-installed on all EC2 instances, which allowed me to manage AWS resources directly from the instance."

The first command I ran was `aws s3 ls`, which lists the S3 buckets in my account.

Next, I ran `aws configure`, which allows you to set up your AWS CLI credentials, including your access key, secret key, default region, and output format.

```
[ec2-user@ip-10-0-4-207 ~]$ aws s3 ls
Unable to locate credentials. You can configure credentials by running "aws login".
[ec2-user@ip-10-0-4-207 ~]$ aws configure
AWS Access Key ID [None]:
```

Access keys

Credentials

To configure my EC2 instance to interact with the broader AWS environment, I ran `aws configure`, which prompted me to enter my access key, secret key, and default region. Providing these credentials allowed the CLI to authenticate properly and establish a connection to my S3 bucket.

Access keys are credentials that allow applications (S3) and other servers to log in to AWS and interact with my resources.

A secret access key is the password that pairs with my access key ID. I need both keys together to be able to access my AWS services, and anyone who obtains them can gain access to my AWS account.

Best practice

While I used access keys for this project to demonstrate the underlying mechanics of AWS CLI authentication, I understand that a best practice alternative in production environments is to create an IAM role with the necessary permissions and attach it directly to the EC2 instance. This approach is generally preferred because it eliminates the need to store long-lived credentials on the instance, and AWS automatically handles credential rotation behind the scenes, reducing the risk of credential leakage or misuse.

In the second part of my project...

Step 4 - Set up an S3 bucket

In this step, I will instance will connect to, demonstrating how compute resources within a VPC can interact with storage resources that exist outside of it.

Step 5 - Connecting to my S3 bucket

"In this step, I returned to my EC2 instance and used my access keys to authenticate and connect with my S3 bucket, allowing me to demonstrate how resources outside the VPC, like S3, can be securely accessed using AWS CLI credentials.

Connecting to my S3 bucket

The first command I ran was `aws s3 ls`, which lists the S3 buckets in my account.

When I ran `aws s3 ls` again, the terminal returned `mel-vpc-project`, confirming that the EC2 instance successfully authenticated and established a connection to my S3 bucket.

```
Last login: Thu Jun 18 02:14:05 2026 from 18.206.107.27
[ec2-user@ip-10-0-4-207 ~]$ aws s3 ls
2026-06-18 02:03:01 mel-vpc-project
[ec2-user@ip-10-0-4-207 ~]$
```

Connecting to my S3 bucket

"I also ran `aws s3 ls s3://mel-vpc-project`, a more specific command that returned the names of both files I had previously uploaded, confirming that the EC2 instance had full visibility into the contents of the S3 bucket.

```
Last login: Thu Jun 18 02:14:05 2026 from 18.206.107.27
[ec2-user@ip-10-0-4-207 ~]$ aws s3 ls
2026-06-18 02:03:01 mel-vpc-project
[ec2-user@ip-10-0-4-207 ~]$ aws s3 ls s3://mel-vpc-project
2026-06-18 02:05:17      1598726 kari-shea-1SAnrIw50Y-unsplash.jpg
2026-06-18 02:05:23      1825778 lorenzo-herrera-p0j-mE6mGo4-unsplash.jpg
[ec2-user@ip-10-0-4-207 ~]$
```

Uploading objects to S3

Before uploading a new file to my bucket, I first ran `sudo touch /tmp/test.txt` on my EC2 instance, which created an empty text file to serve as a test object for the upload process.

The second command I ran was `aws s3 cp /tmp/test.txt s3://mel-vpc-project`, which uploaded a local file from the EC2 instance directly into my S3 bucket, demonstrating the ability to push data from compute resources into external storage." One thing worth double-checking: is `test.txt` actually one of the two files you mentioned uploading earlier from your local computer, or is this a separate test file you created directly on the EC2 instance? That distinction might matter for the accuracy of your write-up.

I then ran `aws s3 ls s3://mel-vpc-project` again to confirm that both uploaded files were still present in the bucket, verifying that the connection between the EC2 instance and S3 remained consistent.

```
[ec2-user@ip-10-0-4-207 ~]$ sudo touch /tmp/test.txt
[ec2-user@ip-10-0-4-207 ~]$ aws s3 cp /tmp/test.txt s3://mel-vpc-project
upload: ../../tmp/test.txt to s3://mel-vpc-project/test.txt
[ec2-user@ip-10-0-4-207 ~]$ aws s3 ls s3://mel-vpc-project
2026-06-18 02:05:17      1598726 kari-shea-1SAnrIw50Y-unsplash.jpg
2026-06-18 02:05:23      1825778 lorenzo-herrera-p0j-mE6mGo4-unsplash.jpg
2026-06-18 03:08:26           0 test.txt
[ec2-user@ip-10-0-4-207 ~]$
```